

AI ASSISTED CODING

ASSIGNMENT-8.5

NAME:DANCHANALA PAVAN

ROLL NO:2403A52L07

BATCH:50

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.

- Requirements:

- o Username length must be between 5 and 15 characters.

- o Must contain only alphabets and digits.

- o Must not start with a digit.

- o No spaces allowed.

Example Assert Test Cases:

```
assert is_valid_username("User123") == True
```

```
assert is_valid_username("12User") == False
```

```
assert is_valid_username("Us er") == False
```

Expected Output #1:

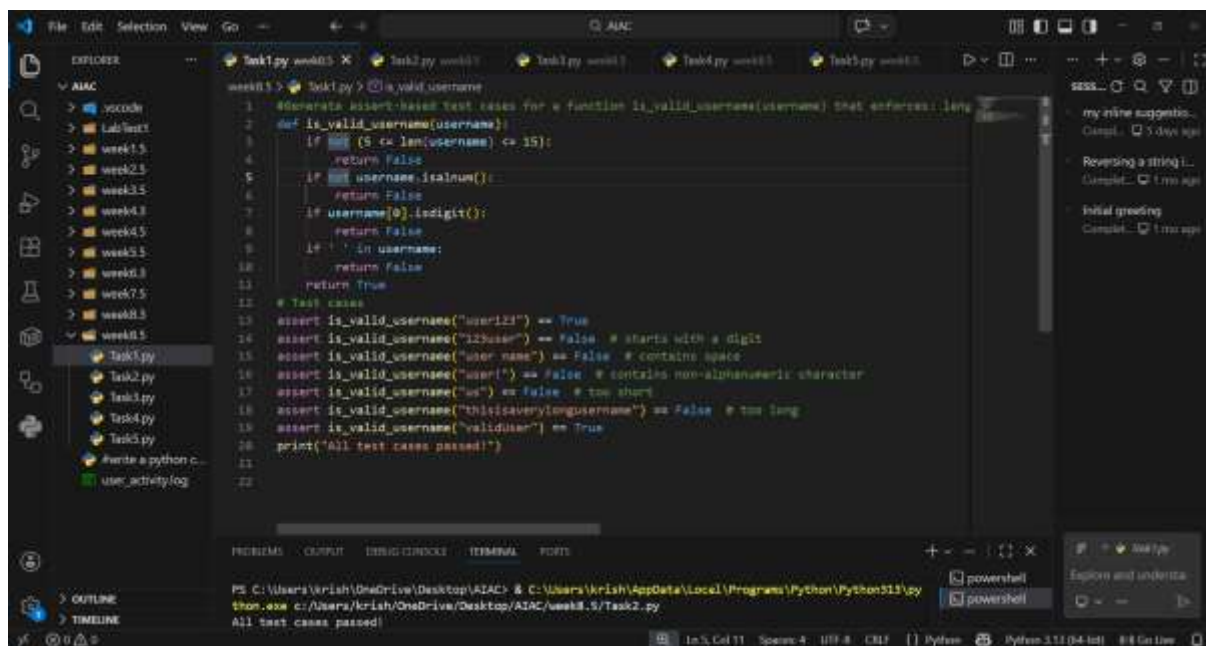
- Username validation logic successfully passing all AI-

generated test cases.

PROMPT:

“Generate assert-based test cases for a function `is_valid_username(username)` that enforces: length between 5–15 characters, only alphanumeric characters, must not start with a digit, and no spaces allowed”

CODE AND OUTPUT:



```
1 #Generate assert-based test cases for a function is_valid_username(username) that enforces: length
2 def is_valid_username(username):
3     if not (5 <= len(username) <= 15):
4         return False
5     if not username.isalnum():
6         return False
7     if username[0].isdigit():
8         return False
9     if ' ' in username:
10        return False
11    return True
12
13 # Test cases
14 assert is_valid_username("user123") == True
15 assert is_valid_username("123user") == False # starts with a digit
16 assert is_valid_username("user name") == False # contains space
17 assert is_valid_username("user!") == False # contains non-alphanumeric character
18 assert is_valid_username("us") == False # too short
19 assert is_valid_username("thisisaverylongusername") == False # too long
20 assert is_valid_username("validuser") == True
21 print("All test cases passed!")
22
```

Terminal Output:

```
PS C:\Users\krish\OneDrive\Desktop\AIAC> & C:\Users\krish\AppData\Local\Programs\Python\Python311\py
thon.exe c:/Users/krish/OneDrive/Desktop/AIAC/week8.5/Task2.py
All test cases passed!
```

EXPLANATION:

We first define assert tests covering valid/invalid usernames. The function checks length (5–15), alphanumeric only, no spaces, and first character not a digit. Regex or string methods can be used. TDD ensures code passes all tests.

Task Description #2 (Even–Odd & Type Classification – Apply

AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.
- Requirements:

o If input is an integer, classify as "Even" or "Odd".

o If input is 0, return "Zero".

o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

```
assert classify_value(8) == "Even"
```

```
assert classify_value(7) == "Odd"
```

```
assert classify_value("abc") == "Invalid Input"
```

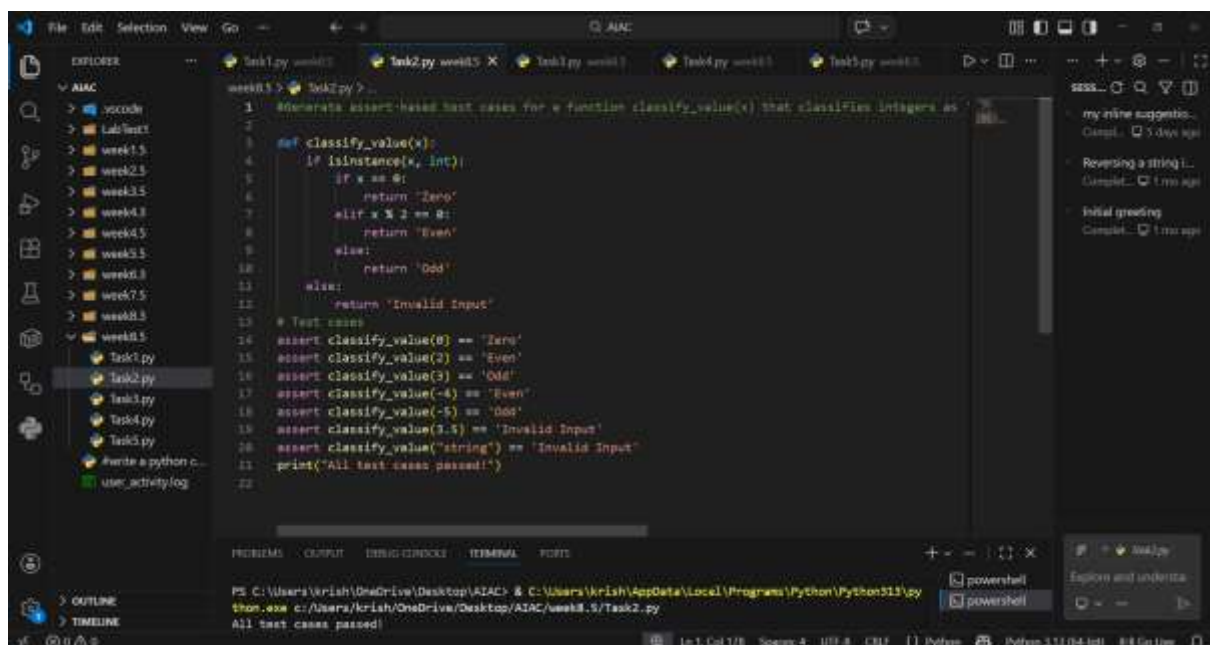
Expected Output #2:

- Function correctly classifying values and passing all test cases.

PROMPT:

"Generate assert-based test cases for a function `classify_value(x)` that classifies integers as 'Even' or 'Odd', returns 'Zero' for 0, and 'Invalid Input' for non-numeric values."

CODE AND OUTPUT:



The screenshot shows a VS Code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor displays a Python script named `Task2.py` with the following content:

```
1 #Generate assert-based test cases for a function classify_value(x) that classifies integers as
2
3 def classify_value(x):
4     if isinstance(x, int):
5         if x == 0:
6             return "Zero"
7         elif x % 2 == 0:
8             return "Even"
9         else:
10            return "Odd"
11    else:
12        return "Invalid Input"
13
14 # Test cases
15 assert classify_value(0) == "Zero"
16 assert classify_value(2) == "Even"
17 assert classify_value(3) == "Odd"
18 assert classify_value(-4) == "Even"
19 assert classify_value(-5) == "Odd"
20 assert classify_value(3.5) == "Invalid Input"
21 assert classify_value("string") == "Invalid Input"
22 print("All test cases passed!")
```

The terminal at the bottom shows the command to run the script and the output:

```
PS C:\Users\krish\OneDrive\Desktop\AIAC> & C:\Users\krish\AppData\Local\Programs\Python\Python311\python.exe c:/Users/krish/OneDrive/Desktop/AIAC/week8.5/Task2.py
All test cases passed!
```

EXPLANATION:

Tests include integers, zero, non-integers, and strings. The function uses type checking, then modulo for even/odd. Zero is handled separately. Non-numeric inputs return "Invalid Input". TDD ensures robustness.

```
def classify_value(x): if isinstance(x, int): if x == 0: return "Zero" return "Even" if x % 2 == 0
else "Odd" return "Invalid Input"
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- **Task:** Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.

- **Requirements:**

- o Ignore case, spaces, and punctuation.

- o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

```
assert is_palindrome("Madam") == True
```

```
assert is_palindrome("A man a plan a canal Panama") ==
True
```

```
assert is_palindrome("Python") == False
```

Expected Output #3:

- **Function correctly identifying palindromes and passing all**

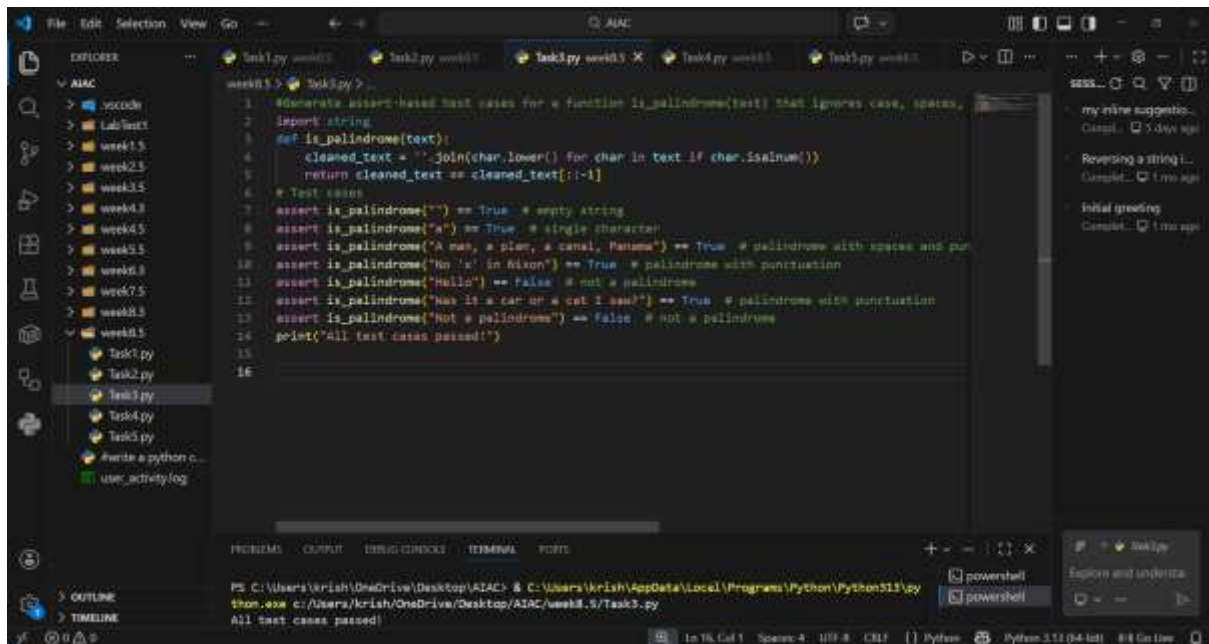
AI-generated tests.

PROMPT:

“Generate assert-based test cases for a function `is_palindrome(text)` that ignores case, spaces, and punctuation, and correctly handles empty strings and single characters.

```
import string”
```

CODE AND OUTPUT:



The screenshot shows a VS Code editor with a Python file named `Task3.py`. The code implements a function `is_palindrome(text)` that checks if a string is a palindrome, ignoring case, spaces, and punctuation. The function uses `re.sub` to clean the string and then compares it with its reverse. The code includes several assert-based test cases for various inputs, and a final print statement indicating that all test cases passed.

```
1 #Generate assert-based test cases for a function is_palindrome(text) that ignores case, spaces,
2 import string
3 def is_palindrome(text):
4     cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
5     return cleaned_text == cleaned_text[::-1]
6 # Test cases
7 assert is_palindrome("") == True # empty string
8 assert is_palindrome("a") == True # single character
9 assert is_palindrome("A man, a plan, a canal, Panama") == True # palindrome with spaces and pun
10 assert is_palindrome("No 'x' in Nixon") == True # palindrome with punctuation
11 assert is_palindrome("Hello") == False # not a palindrome
12 assert is_palindrome("Was it a car or a cat I saw?") == True # palindrome with punctuation
13 assert is_palindrome("Not a palindrome") == False # not a palindrome
14 print("All test cases passed!")
15
```

The terminal output at the bottom shows the command to run the script and the result: `All test cases passed!`

EXPLANATION:

Tests cover case-insensitivity, ignoring spaces/punctuation, empty strings, and single characters. The function normalizes input by removing non-alphanumeric and lowercasing. Then compares string with its reverse. TDD ensures correctness.

Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)

- Task: Ask AI to generate at least 3 assert-based test cases for a `BankAccount` class and then implement the class.

- Methods:

- o `deposit(amount)`

- o `withdraw(amount)`

- o `get_balance()`

Example Assert Test Cases:

```
acc = BankAccount(1000)
```

```
acc.deposit(500)
```


- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.

- Requirements:

- o Must contain `@` and `.`

- o Must not start or end with special characters.

- o Should handle invalid formats gracefully.

Example Assert Test Cases:

```
assert validate_email("user@example.com") == True
```

```
assert validate_email("userexample.com") == False
```

```
assert validate_email("@gmail.com") == False
```

Expected Output #5:

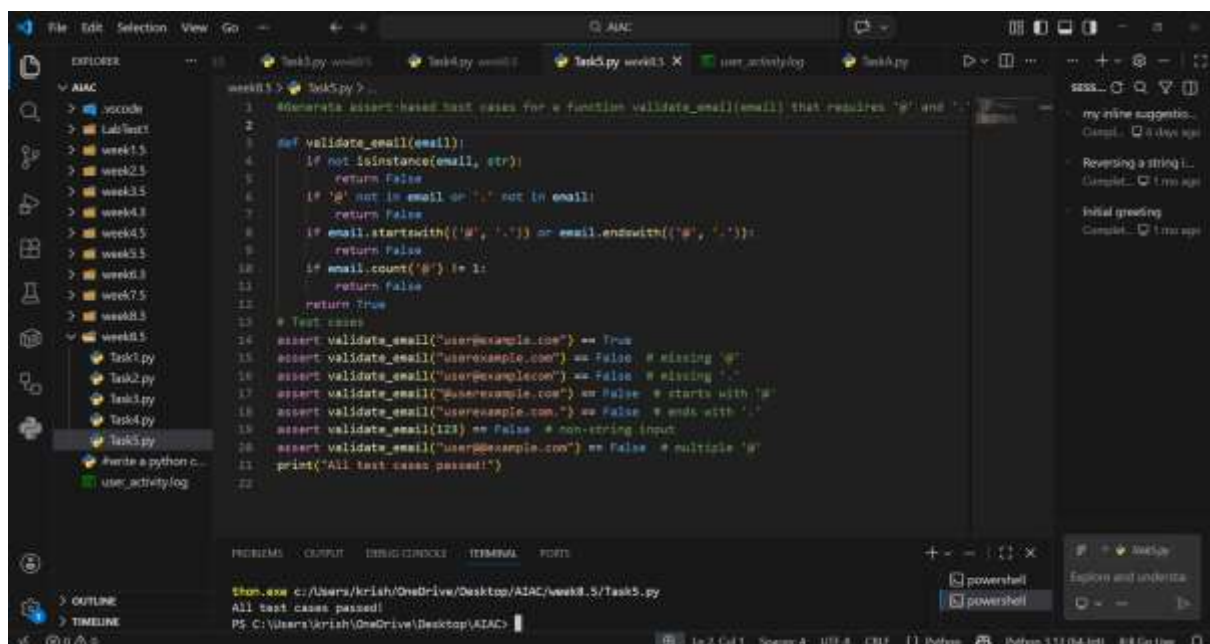
- Email validation function passing all AI-generated test cases

and handling edge cases correctly

PROMPT:

"Generate assert-based test cases for a function `validate_email(email)` that requires '@' and '.', disallows starting/ending with special characters, and handles invalid formats gracefully."

CODE AND OUTPUT:



The screenshot shows a VS Code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor contains a Python script named `Task5.py` with the following content:

```
1 #Generate assert-based test cases for a function validate_email(email) that requires '@' and '.'
2
3 def validate_email(email):
4     if not isinstance(email, str):
5         return False
6     if '@' not in email or '.' not in email:
7         return False
8     if email.startswith(('@', '.')) or email.endswith(('@', '.')):
9         return False
10    if email.count('@') != 1:
11        return False
12    return True
13
14 # Test cases
15 assert validate_email("user@example.com") == True
16 assert validate_email("userexample.com") == False # missing '@'
17 assert validate_email("user@example.co") == False # missing '.'
18 assert validate_email("@userexample.com") == False # starts with '@'
19 assert validate_email("userexample.com.") == False # ends with '.'
20 assert validate_email(123) == False # non-string input
21 assert validate_email("User@example.com") == False # multiple '@'
22 print("All test cases passed!")
```

The terminal at the bottom shows the command `python Task5.py` being executed, resulting in the output `All test cases passed!`.

EXPLANATION:

Tests include valid emails, missing symbols, invalid domains, and edge cases. The function checks presence of @ and ., ensures no leading/trailing special characters, and validates format. TDD ensures graceful handling.